Прототип сервиса

1. Разработка интерфейса с использованием Gradio

Для создания пользовательского интерфейса использована библиотека Gradio, которая предоставляет простой и удобный способ создания интерфейсов для приложений машинного обучения. Интерфейс состоит из следующих элементов:

- **Загрузка фотографии** — пользователь может загрузить изображение, которое будет использовано для генерации аватара.
- **Предварительный просмотр аватара без фона** — после обработки изображения пользователю отображается результат в виде аватара без фона.
- **Кнопка для создания аватара** — запуск процесса обработки изображения, включающего удаление фона и создание 3D-модели.
- **Опция для скачивания 3D-модели** — после завершения обработки доступна возможность скачать "3D-модель" в формате .obj, хотя на текущем этапе это всего лишь заглушка.

```
import gradio as gr

with gr.Blocks() as demo:
gr.Markdown("## 👤 Генератор 3D-Аватара из фото (демо)")
with gr.Row():
img_input = gr.Image(type="pil", label="Загрузите фото")
img_output = gr.Image(label="Предварительный аватар (без фона)")
with gr.Row():
btn = gr.Button("Создать аватар")
file_download = gr.File(label="⬇️ Скачать 3D-модель (.obj)")
btn.click(generate_avatar, inputs=img_input, outputs=[img_output,
file_download])

demo.launch()
```

2. Удаление фона с использованием библиотеки rembg

Одним из основных этапов обработки изображений является удаление фона, что достигается с помощью библиотеки **rembg**. Эта библиотека использует современные алгоритмы для сегментации изображений, эффективно отделяя

объект от фона. После обработки изображения результат сохраняется в формате PNG и предоставляется пользователю.

```
from rembg import remove
from PIL import Image
import os

output_dir = "avatars"
os.makedirs(output_dir, exist_ok=True)

def generate_avatar(image):
    result = remove(image)  # удаление фона
    avatar_path = os.path.join(output_dir, "avatar.png")
    result.save(avatar_path)

    # ЗАГЛУШКА: создание "3D модели"
    fake_3d_path = os.path.join(output_dir, "avatar_model.obj")
    with open(fake_3d_path, "w") as f:
        f.write("# Здесь могла быть 3D-модель")

    return avatar_path, fake_3d_path
```

3. Создание "3D-модели" (заглушка)

В рамках прототипа реализована заглушка для 3D-модели, которая представляет собой пустой файл .obj. Этот файл создается с помощью записи простого текста, указывающего на отсутствие реальной 3D-модели. В дальнейшем предполагается интеграция с реальными 3D-генераторами, которые смогут создавать полноценные 3D-модели на основе загруженных изображений.

```
fake_3d_path = os.path.join(output_dir, "avatar_model.obj")
with open(fake_3d_path, "w") as f:
    f.write("# Здесь могла быть 3D-модель")
```

4. Организация сохранения и скачивания файлов

Все сгенерированные аватары и 3D-модели сохраняются в папке **avatars**, что упрощает организацию хранения файлов. Папка автоматически создается при запуске приложения, если она не существует. Это обеспечивает удобство работы с сервисом и упрощает доступ к файлам для их скачивания.

```
output_dir = "avatars"
os.makedirs(output_dir, exist_ok=True)
```

5. Структура проекта

Структура проекта была спроектирована с учетом будущей масштабируемости и удобства работы:

- **app.py** — основной файл с кодом приложения.
- **requirements.txt** — файл с зависимостями, необходимыми для корректной работы приложения.
- **avatars/** — папка для хранения сгенерированных аватаров и 3D-моделей.
- **README.md** — файл с описанием проекта и инструкциями по его запуску.

Полный код app.py

```
import gradio as gr
from rembg import remove
from PIL import Image
import os
import shutil

output_dir = "avatars"
os.makedirs(output_dir, exist_ok=True)

def generate_avatar(image):
    result = remove(image)  # убираем фон
    avatar_path = os.path.join(output_dir, "avatar.png")
    result.save(avatar_path)

    # ЗАГЛУШКА: создаём "3D модель" (будто .obj-файл)
    fake_3d_path = os.path.join(output_dir, "avatar_model.obj")
    with open(fake_3d_path, "w") as f:
        f.write("# Здесь могла быть 3D-модель")  # В будущем — настоящий
obj

    return avatar_path, fake_3d_path

with gr.Blocks() as demo:
    gr.Markdown("## 👤 Генератор 3D-Аватара из фото (демо)")
    with gr.Row():
        img_input = gr.Image(type="pil", label="Загрузите фото")
        img_output = gr.Image(label="Предварительный аватар (без фона)")
    with gr.Row():
        btn = gr.Button("Создать аватар")
        file_download = gr.File(label="⬇️ Скачать 3D-модель (.obj)")
```

```
    btn.click(generate_avatar, inputs=img_input, outputs=[img_output,
file_download])

demo.launch()
```

requirements.txt

```
gradio
rembg
Pillow
onnxruntime
```

README.md

```
# Генератор 3D-Аватара из Фото

Этот проект представляет собой прототип онлайн-сервиса для генерации
аватаров и "3D-моделей" (в формате .obj) из загруженных фотографий.
Сервис использует технологию удаления фона с изображений, а также создает
заглушку для 3D-модели. Проект реализован с использованием Python и
библиотеки Gradio.

## Описание

Сервис позволяет пользователю загрузить фото, после чего:
1. Удаляется фон с изображения.
2. Генерируется аватар без фона в формате PNG.
3. Создается заглушка для 3D-модели в формате .obj (на данный момент
модель не является полноценной 3D-моделью, а просто текстовым файлом).

## Структура проекта

Проект состоит из следующих файлов и папок:
- `app.py` — основной файл, который запускает приложение и обрабатывает
изображения.
- `requirements.txt` — файл с перечнем зависимостей, необходимых для
работы приложения.
- `avatars/` — папка для хранения сгенерированных аватаров и 3D-моделей.
- `README.md` — описание проекта.

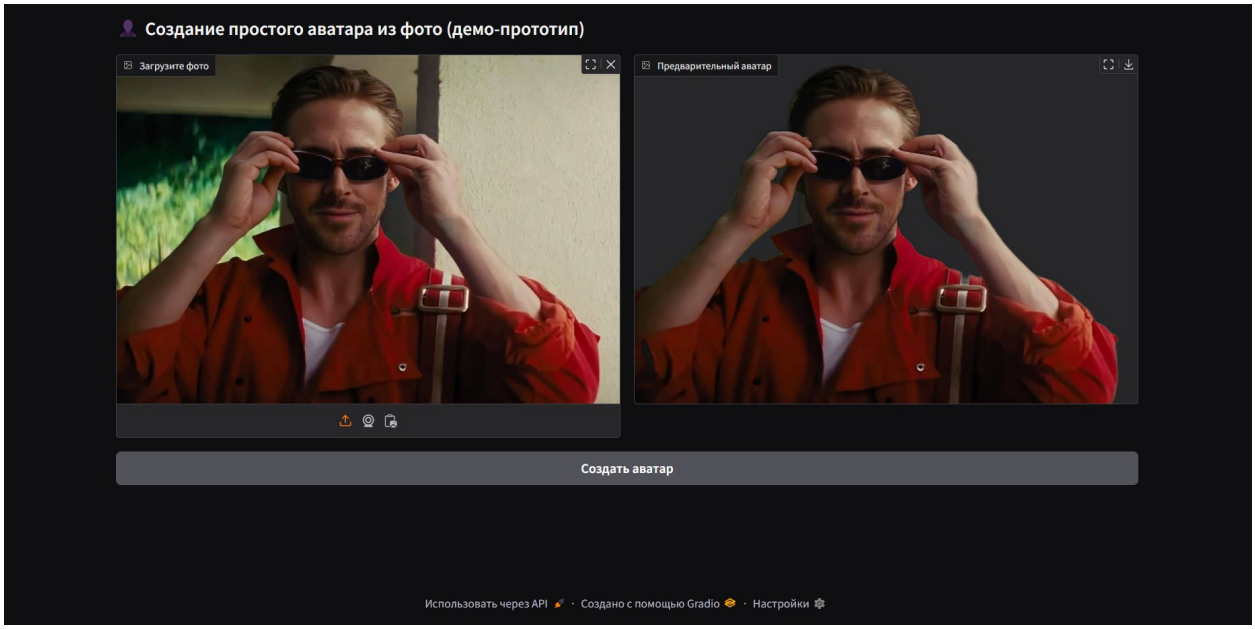
## Установка

1. Установить зависимости:
    ```bash
    pip install -r requirements.txt
    ```
```

```
## Запуск

Для запуска приложения выполните команду:
```bash
python app.py
```

Промежуточный результат



Перспективы и дальнейшее развитие

На текущий момент прототип выполняет базовую задачу: удаляет фон с изображения и генерирует заглушку для 3D-модели. В дальнейшем предполагается:

1. **Интеграция с настоящими 3D-генераторами**, что позволит создавать полноценные 3D-модели на основе загруженных изображений.
2. **Улучшение и оптимизация пользовательского интерфейса** под выбранные 3D генераторы.
```